

ECE 1387 Lecture #9

Nov. 14, 2025

Partitioning cont'd...

Now...

Scalable partitioning

- heuristic
- can apply to
huge graph, say 100K
vertices.

FM Partitioning
↓
Fiducia → Matthyses

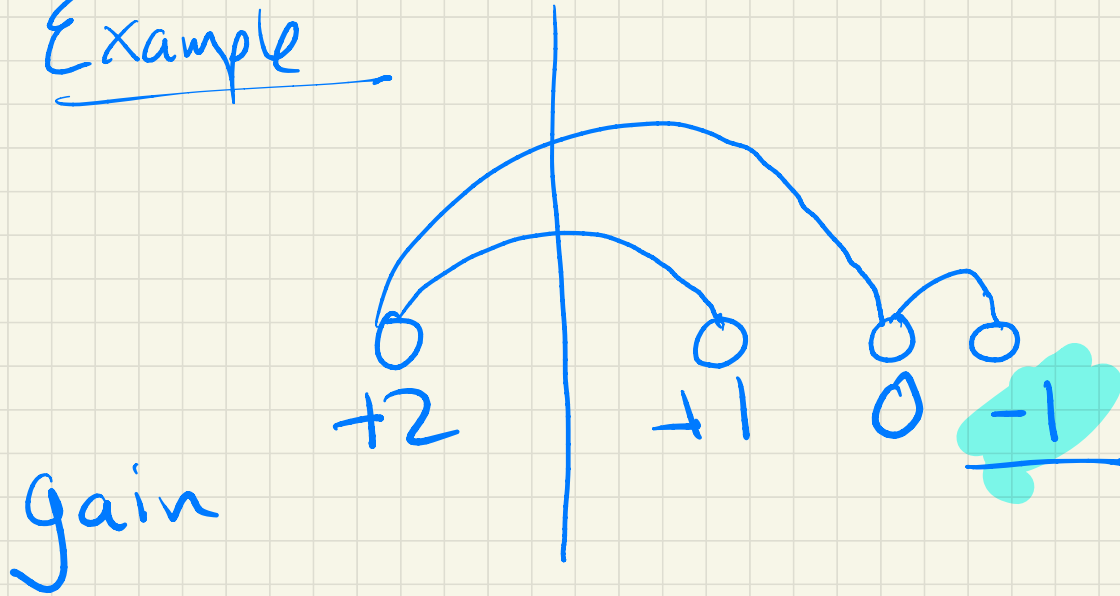
- DAC '82
- General Electric
- FM

Definition **gain** of a vertex v , $\text{gain}(v)$

- Reduction in cut size

(crossing count) if vertex
 v is moved to opposite
partition (bipartitioning)

Example



negative gain means an increase
in crossing count.

FM Algorithm

Given: initial balanced partition V_1, V_2

Repeat:

Unlock all vertices

Compute initial gain for all verts.

for $i = 1$ to n // $n = |V|$

Choose a free (not locked)

vertex b s.t. b has
highest gain and moving b
preserves balance

- if no such b exists
break

- move b and lock it

- $g_i = \text{gain}(b)$

~ update the affected cell gains *

end for

Find k s.t. $G = g_1 + g_2 + \dots + g_k$
is maximized

Shuffle cells upto k^{th} step

Until $G \leq 0$

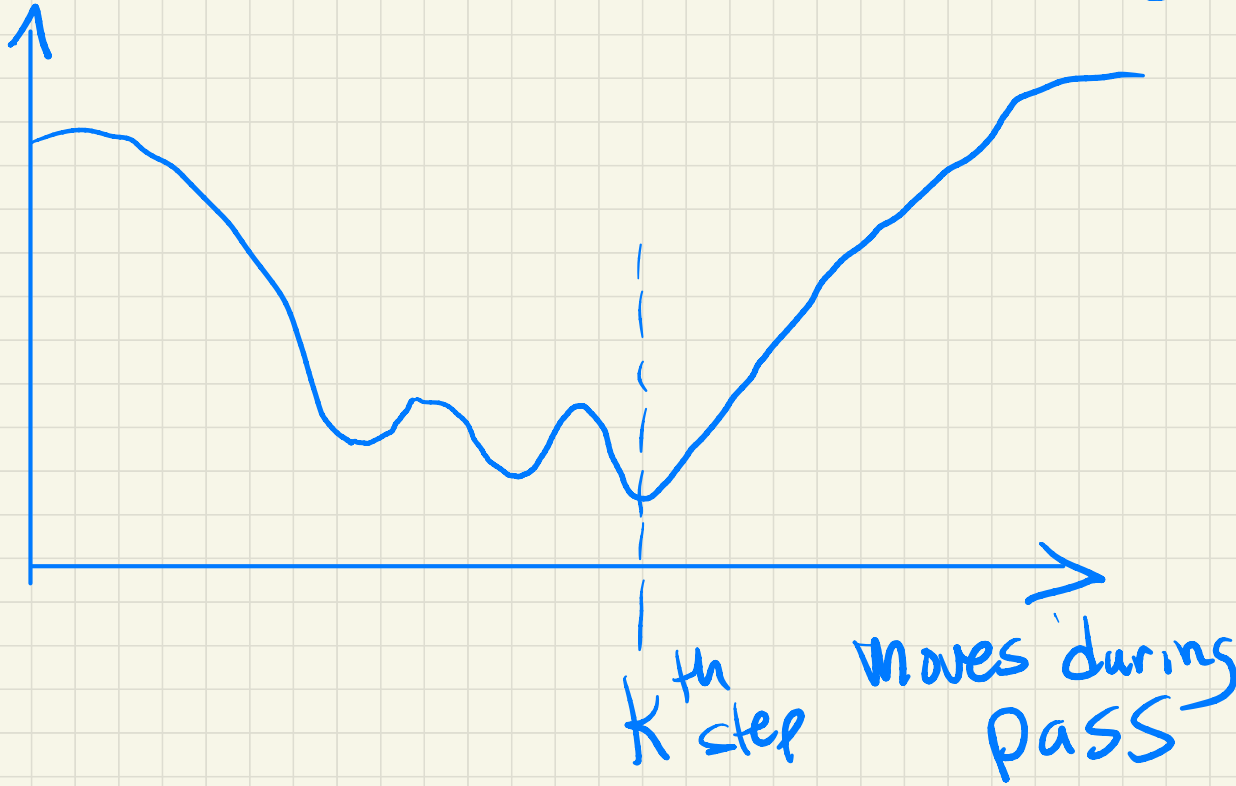
each iter of
repeat is
one pass

~~if~~ only cells connected to b
could potentially need gain
updates.

— Each pass begins with best
partitioning from prev. pass *

cutsize

FM allows hill climbing



Main breakthrough in FM

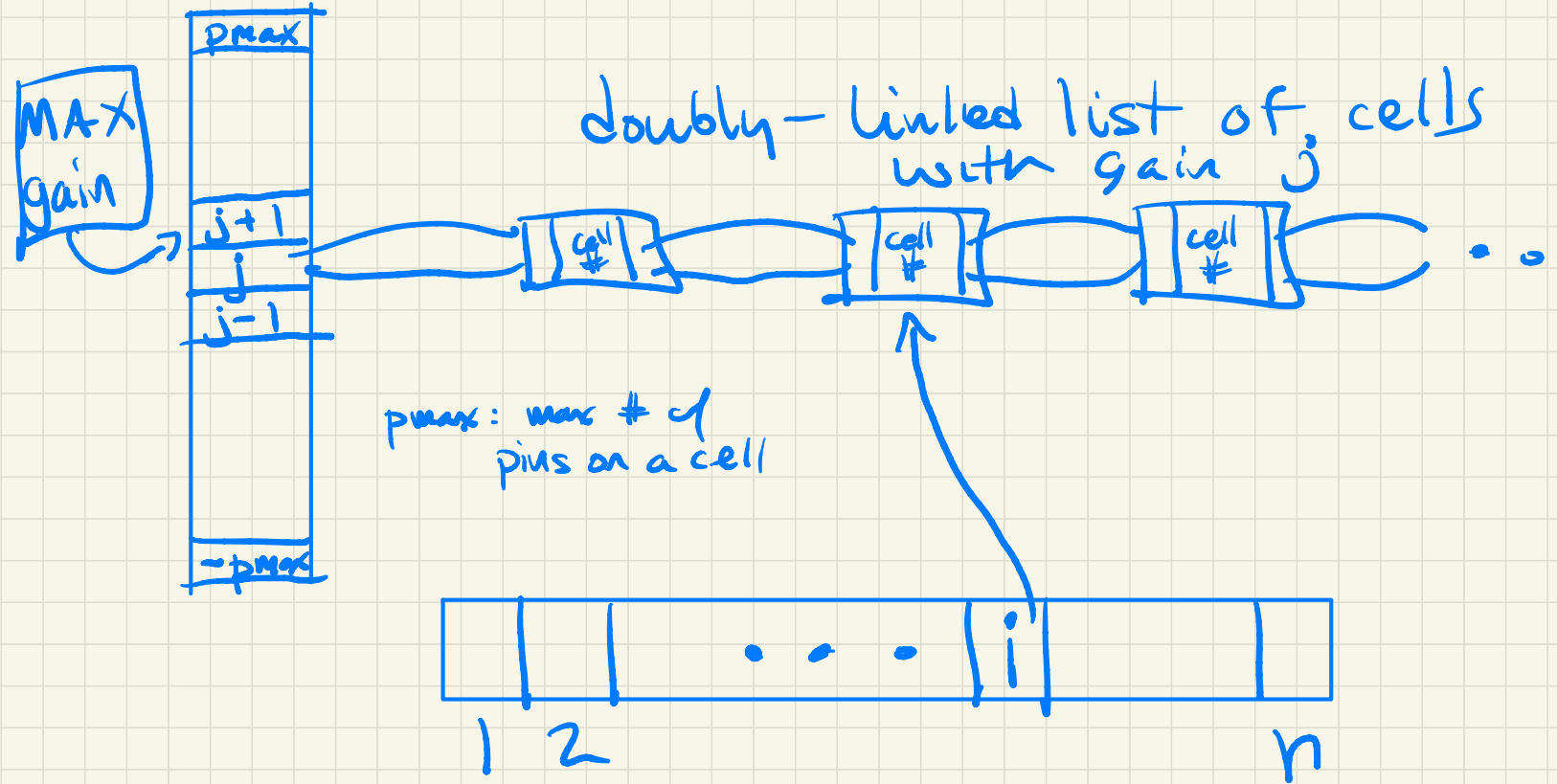
- Let $P = \#$ of pins in circuit (graph)

- Each pass takes $O(P)$ time

↳ linear time!

- Using clever data structure + reasoning

Gain bucket data structure

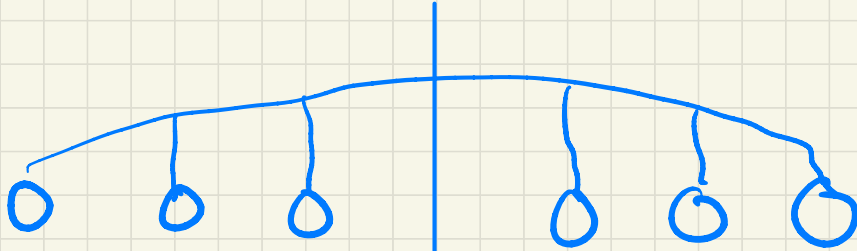


One gain bucket data structure
for V_1 , one for V_2

- Find a cell max gain in $O(1)$
time
- Remove/insert a cell in $O(1)$
time
- Update gain(i) for cell i in
 $O(1)$ time
- Update max gain pointer in
 $O(p)$ time // see paper

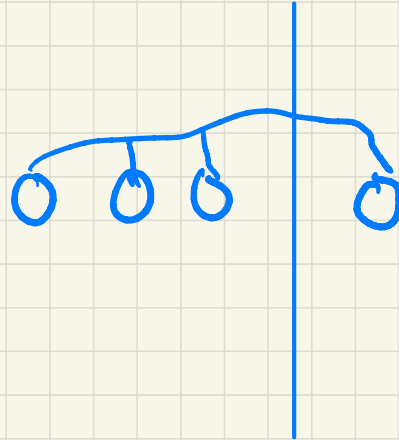
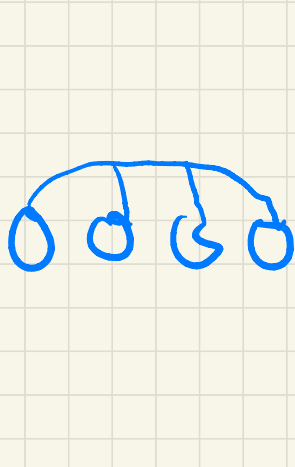
Key observation: when a cell b is moved, not all cells connected to b necessarily need gain updates.

Non critical net



has > 1 vertex in both partitions

Critical net has 0 or 1 vertex
in one partition. (V_1 or V_2)



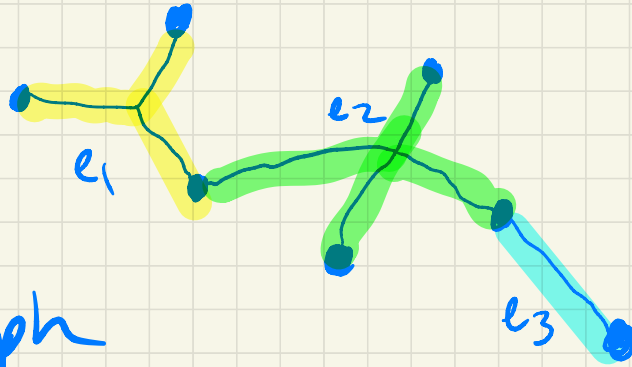
- Only nets that are critical before or after a move cause gain updates to their cells.
- Once a cell on net n moves from $V_1 \rightarrow V_2$ and another cell on net n moves from $V_2 \rightarrow V_1$, net n is "dead"
- ↳ dead nets do not trigger gain updates

- Every net will bring about changes to the gains of its cells at most K times ($K=4$)
- $O(P)$ gain updates/pass!

hMetis Multi-level Partitioning

IEEE TVLSI
199

Univ of Minnesota



$h = \text{hypergraph}$

↳ edges can have
 > 2 nodes

Size of edge =
nodes on edge

$$|e_1| = 3 \quad |e_2| = 4 \quad |e_3| = 2$$

An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design

Ismail Bustany[†], Grigor Gasparyan[†], Andrew B. Kahng, Ioannis Koutis[‡], Bodhisatta Pramanik, Zhiang Wang

University of California San Diego, La Jolla, CA, USA

[†]Advanced Micro Devices, San Jose, CA, USA

[‡]New Jersey Institute of Technology, Newark, NJ, USA

Email: [†]{ismail.bustany, grigor.gasparyan}@amd.com, {abk, bopramanik, zhw033}@ucsd.edu, [‡]ikoutis@njit.edu

Abstract—With the increasing complexity of IC products, large-scale designs must be efficiently partitioned into multiple blocks, tiles, or devices for concurrent backend place-and-route (P&R) implementation. State-of-the-art partitioners focus on balanced min-cut without considering constraints such as timing or heterogeneity of resource types. They are thus increasingly unsuitable for current physical design requirements. We introduce *TritonPart*, the first open-source, constraints-driven partitioning tool for VLSI physical design. *TritonPart* employs efficient algorithms to handle constraints, including multi-dimensional balance, embedding, and timing constraints. Our experimental work affirms its benefits. For standard min-cut partitioning, *TritonPart* outperforms *hMETIS* [17], with improvements of up to ~20% on some benchmarks. For embedding-aware partitioning, *TritonPart* effectively leverages the embeddings generated by *SpecPart* [4] and improves upon it by ~2%. For timing-aware partitioning, *TritonPart* significantly reduces the number of cuts on timing-critical paths and prevents timing-noncritical paths from becoming critical (~21X, ~119X reduction relative to *hMETIS* and *KaHyPar* [31], respectively).

Index Terms—hypergraph partitioning, multi-dimensional weights, timing, embedding, VLSI constraints

I. INTRODUCTION

Hypergraph partitioning is a fundamental optimization problem in VLSI CAD. Partitioning is arguably becoming even more crucial due

• *Hypergraph Partitioning with Pragmatic Constraints*

We present a generalized hypergraph partitioning formulation. We extend the standard scalar vertex weights to multi-dimensional vectors, with corresponding multi-dimensional balance constraints [2]. We also include in our formulation constraints that are informed by real applications across various domains. These include timing path cut, fixed vertex, grouping [37], and “soft” embedding constraints.

• *General Partitioning Multi-Tool*

We present *TritonPart*, the first open-source framework that is able to simultaneously honor all the aforementioned constraints. *TritonPart* is released under a permissively open-source license enabling other researchers to readily adapt it to accommodate different constraints [1].

• *Novel Algorithms for Multilevel Partitioning*

TritonPart follows the established multilevel partitioning paradigm, but also incorporates elements from [4]. In addition, *TritonPart* consists of multiple algorithms that enable it to handle constraints, including a novel slack propagation algorithm that enables it to reduce the number of cuts on timing-critical paths and prevent timing-noncritical paths from becoming critical.

Given an input hyper graph

1. Construct a seq a successively smaller graphs "coarsening step"
 - done by collapsing certain nodes together in "super nodes"
2. Apply partitioning to smallest graph -- compute bisection
3. Project partitioning onto the next finer level of graph ("uncoarsening")

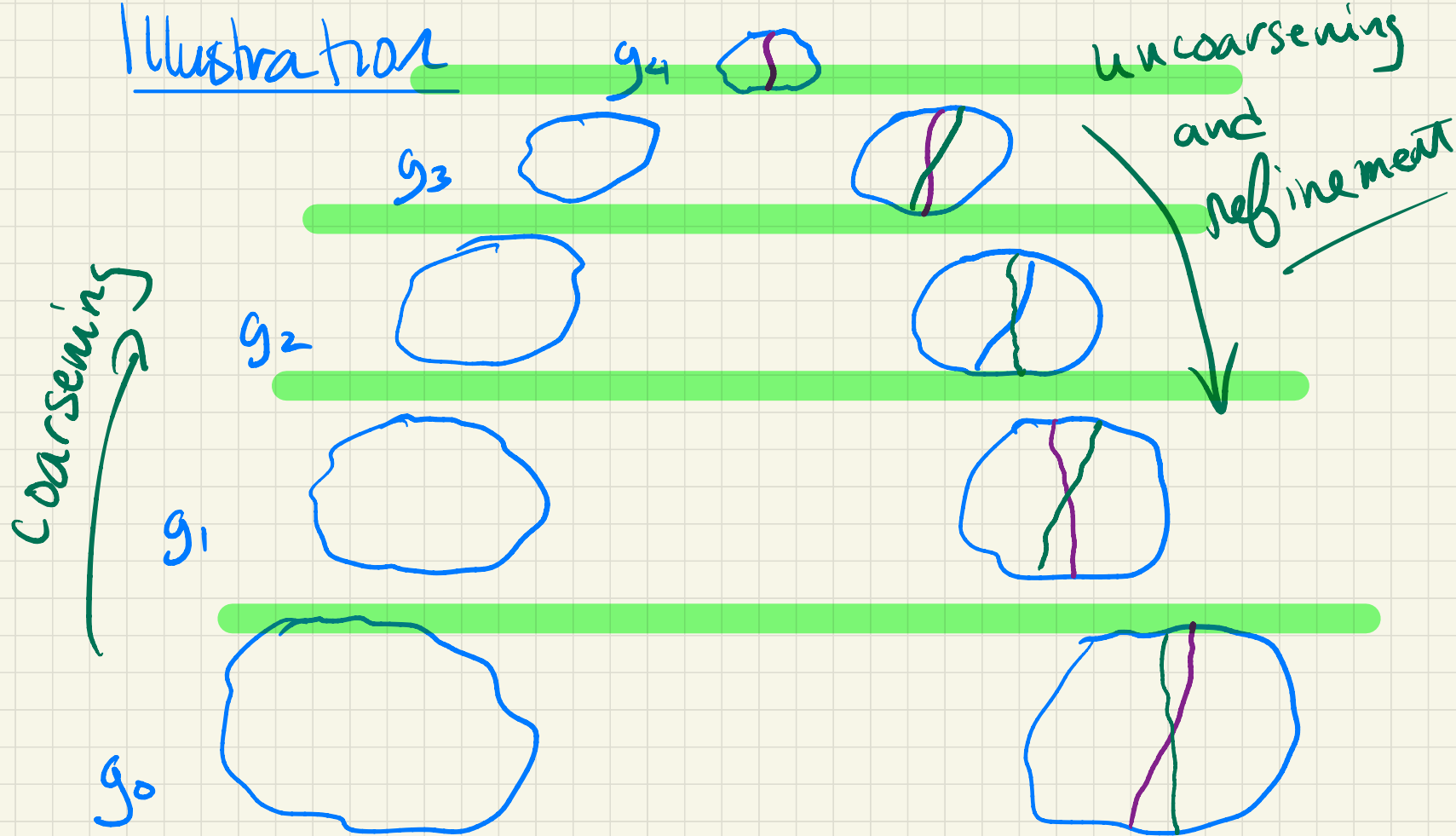
and apply iterative refinement to improve partitioning

- choose some nodes to move to other partition

- a node may correspond to multiple (many) nodes in original graph

#4 Repeat step #3 until we
return to original flat graph

Illustration



Avoids full blown partitioning on
flat graph Go

Pieces of hMetis

1) Coarsening phase

- create a smaller graph
- want a partitioning of smaller graph to be a decent partitioning of larger graph
- collapse connected cells together

3 ways of coarsening

i) edge coarsening (used in Triton Part)

— choose two vertices connected to one another & collapse them

Visit verts in random order

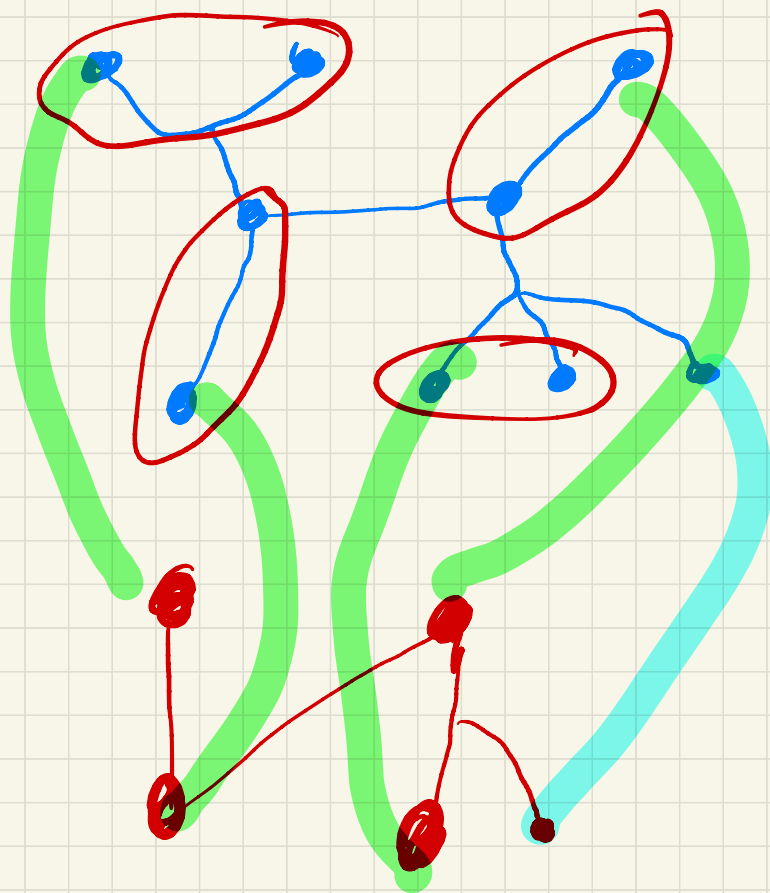
For each uncollapsed vertex v

find vertex u , s.t. u is uncollapsed and (u, v) connection has highest weight

$$\text{weight}(e) = \frac{1}{|e|-1}$$

$\Rightarrow$ low fanout nets have higher weight

— Collapse u, v together



Hyperedge coarsening

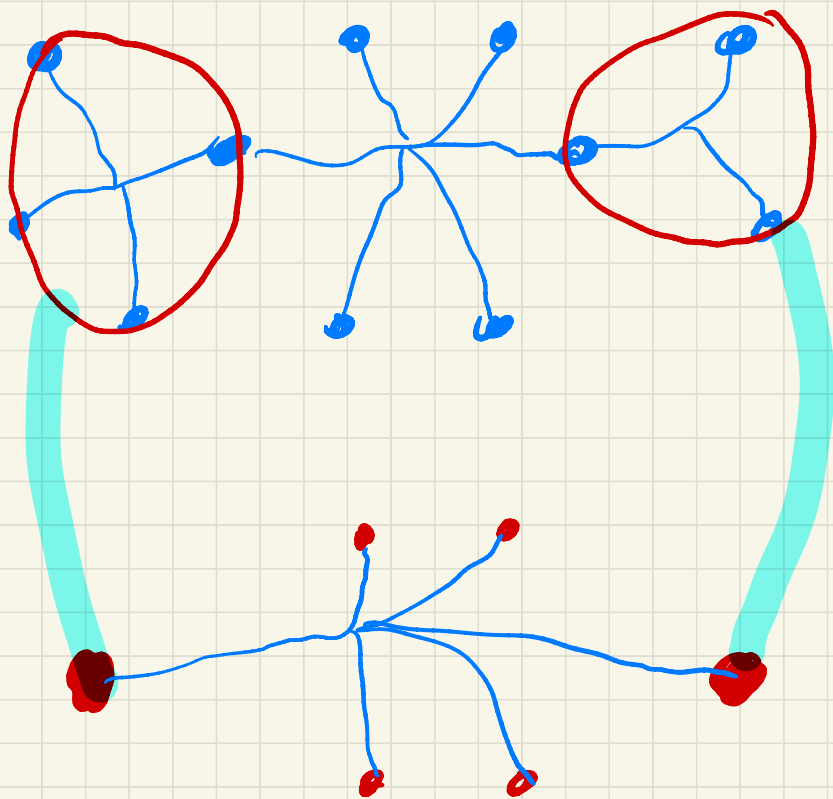
Sort hyperedges in decreasing weight order -

→ randomly for a given weight

Visit hyperedges in order

For each hyperedges that connects vertices that have not been collapsed

— Collapse whole hyperedge

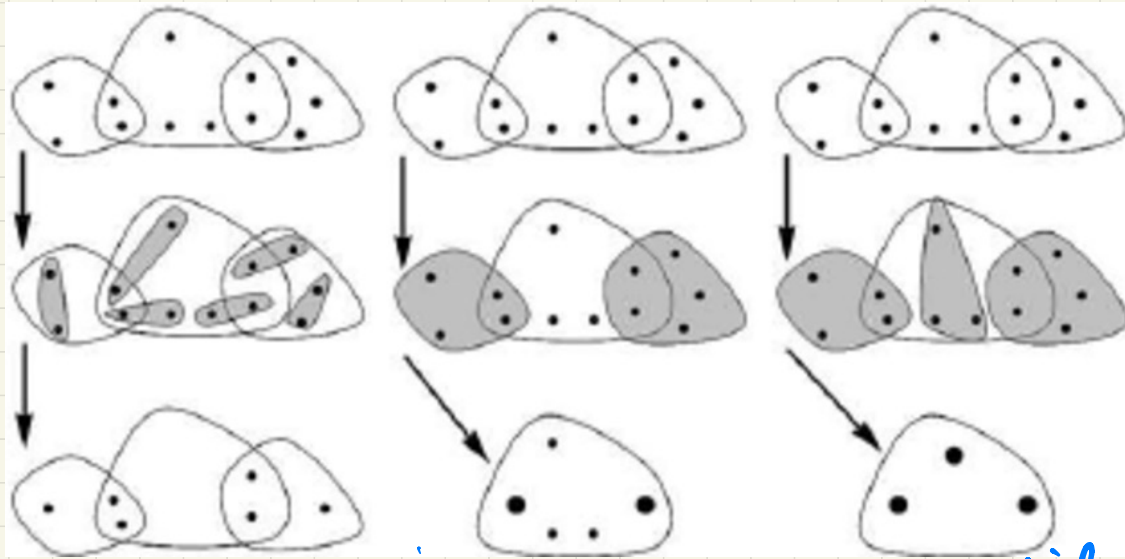


Nodes have
different #s
of internal
nodes

Modified Hyperedge Coarsening

- After hyperedge coarsening
- Traverse hyperedges again
- For each hyperedge not collapsed, collapse its remaining nodes.

Coarsening Options



edge coarsening

hyperedge

modified
hyperedge

https://www.researchgate.net/figure/Coarsening-phase-a-Edge-coarsening-connected-pairs-of-vertices-are-matched_fig1_220781339

Initial partitioning of small graph
(< 200 vertices)

- Two approaches (h Metis)

1) random partitioning

2) BFS from a random vertex

- Other ideas

- B&B

- ILP (integer linear programming)

↳ Triton Part